Satellite Fire Detection Warehouse for Monitoring Agricultural Land Conversion: Progress Report

Shiraj Pradhanang 6836905
Shabirullah Azizi 6938721
Benjamin Fenelon 6936309
ITCS544 Term Project
Faculty of Information and Communication Technology, Mahidol University
Salaya, Thailand

Github link: https://github.com/shirajpng/fire-analysis-dwhm

youtube link: https://www.youtube.com/watch?v=0FN2t0f6uVQ

***Abstract*—This report documents progress on a multi-tier data warehouse for satellite active-fire detections over Thailand in 2024, built from MODIS and VIIRS (S-NPP and NOAA-20) fire archives and conformed against World Database on Protected Areas (WDPA) boundaries. The warehouse runs on PostgreSQL with the PostGIS extension inside Docker and is organized into a staging area, a normalized data store (NDS, 3NF), a dimensional data store (DDS, a star schema with SCD Type 2 protected-area boundaries) and a metadata layer. We describe the logical and physical models, the Python ETL pipeline and its Apache Airflow orchestration with incremental extraction, a PostGIS point-in-polygon spatial join, five benchmarked OLAP queries, and a preliminary mining pipeline using HDBSCAN spatio-temporal clustering and Isolation Forest anomaly detection.**

I. Introduction

Agricultural expansion into forest and savanna is driven by burning. Satellites detect those fires several times a day, and NASA FIRMS publishes every detection as an open CSV. This project builds a multi-tier data warehouse over these detections for a single region, conforms them against protected-area boundaries, and mines the result: clustering individual detections into discrete fire events and detecting anomalous burning inside legally protected areas. The data is genuinely raw: multiple satellites, inconsistent schemas between MODIS and VIIRS, and duplicate detections of the same fire on overlapping overpasses. Our study region is Thailand, using 2024 detections.

II. Logical and Physical Modeling

A. Environment and Data Stores

The warehouse runs on a PostgreSQL database server with the PostGIS extension, deployed with Docker (`Docker-compose.yml`). When the container starts, Docker automatically runs the schema initialization scripts (`init_scripts/01_init_schemas.sql`) and creates four data stores: `stage`, `nds`, `dds` and `meta`. The physical NDS, DDS (star schema) and Meta layers are created by `init-scripts/03_create_nds_dds_meta.sql`.

B. Normalized Data Store (NDS)

The NDS is the 3NF master store. It holds historical transaction details and raw sensor readings from the MODIS, VIIRS and WDPA feeds in five tables:

- `nds_satellite`
- `nds_instrument`
- `nds_location`
- `nds_protected_area`
- `nds_fire_event`

The full Entity-Relationship Diagram is shown in Fig. 1.

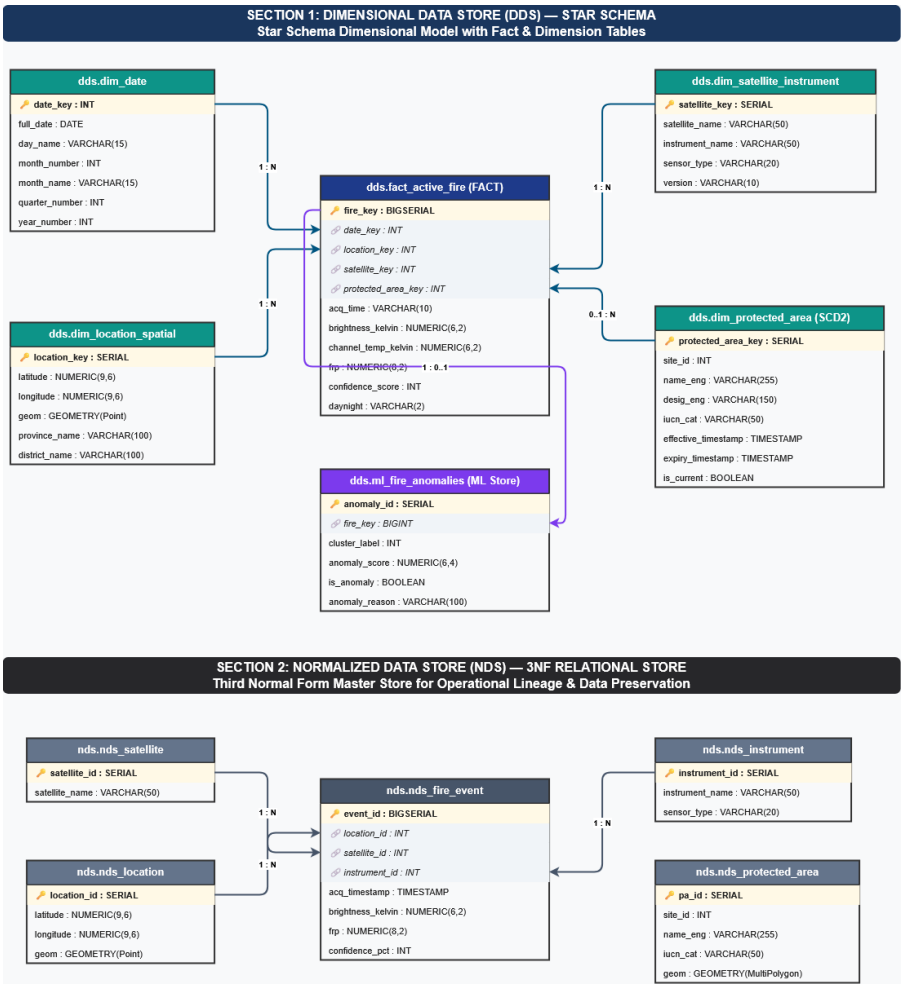


Fig. 1. Entity-Relationship Diagram of the normalized data store.

C. Dimensional Data Store (DDS)

The DDS is a star schema used for OLAP queries. It consists of one fact table and five dimension tables:

- Fact: `fact_active_fire`
- Dimensions: `dim_date`, `dim_location_spatial`, `dim_satellite_instrument`, `dim_protected_area`, `dim_anomaly_classification`

`dim_protected_area` uses SCD Type 2 to track conservation boundaries and manage historical spatial boundary revisions. Its attributes are `protected_area_key`, `site_id`, `name_eng`, `desig_eng`, `iucn_cat`, `rep_area` and `gov_type`, together with the SCD Type 2 versioning columns `effective_timestamp`, `expiry_timestamp` and `is_current`. A default member with key 0 (“Unprotected / Outside PA”) represents detections that fall outside any protected area.

Table I gives the bus matrix relating business processes to the conformed dimensions.

TABLE I
BUS MATRIX

Business Process / Data Mart	dim_date	dim_location_spatial	dim_satellite_instrument	dim_protected_area	dim_anomaly_classification
Active Fire Detection & Monitoring	x	x	x	x	
Conservation Area Vulnerability Analysis	x	x		x	
Multi-Sensor Calibration & Accuracy	x		x		
Machine Learning Anomaly Mining	x	x	x	x	x

D. Metadata Layer

The `meta` schema manages table and column definitions, data mappings and ETL timestamps:

- `meta.ds_table`
- `meta.ds_column`
- `meta.data_mapping`
- `meta.etl_data_flow` (also stores the incremental-extraction timestamps, Section III-E)
- `meta.event_log`

III. ETL PIPELINE EXECUTION

A. Source Data

Raw data are stored from the sources listed in Table II: MODIS and VIIRS active-fire files from NASA FIRMS and the WDPA protected-area attribute table. There are a total of 39,064 rows from the MODIS instrument and 289,864 from the VIIRS instrument. There are also 314,766 rows from the WDPA dataset.

TABLE II
SOURCE DATASETS

File	Source
`modis_2024_Thailand.csv`	MODIS
`viirs-jpss1_2024_Thailand`	VIIRS NOAA-20
`viirs-snpp_2024_Thailand.csv`	VIIRS S-NPP
`fire_archive_M-C61_795810.csv`	MODIS
`fire_archive_SV-C2_795812.csv`	VIIRS S-NPP
`fire_archive_J1V-C2_795811.csv`	VIIRS NOAA-20
`WDPA_Aug2026_Public_csv.csv`	WDPA

B. Staging

The staging schema DDL is defined in `init-scripts/02_create_staging_tables.sql`. The Python script `controllers/etl.py` inserts the datasets into the staging tables.

C. Cleaning and Anomaly Reduction

Anomaly reduction is performed by removing low-confidence rows, as they can be potential false alarms. After loading, the datasets are verified in the database, including a check for redundancies in the longitude and latitude data.

D. NDS and DDS Population

ETL logic in `controllers/etl_dim_fact.py` populates the new NDS, DDS and metadata tables. The default "Unprotected / Outside PA" member of `dds.dim_protected_area` is inserted with a separate SQL statement.

E. Orchestration

An Apache Airflow DAG (`dags/airflow_dag.py`) automates the workflow and manages incremental extraction by tracking the Last Successful Extraction Time (LSET) and Current Extraction Time (CET) in `meta.etl_data_flow`.

IV. Warehouse Benchmarking

A. Spatial Point-in-Polygon Join

Satellite active-fire point locations (`dds.dim_location_spatial`) are connected with conservation boundary polygons (`dds.dim_protected_area`) using a PostGIS spatial join (`ST_Within`), implemented in `init-scripts/04_spatial_join.sql`.

B. OLAP Queries

We execute five analytical OLAP queries to benchmark dimensional performance across temporal, spatial and sensor dimensions (Table III). The benchmark is run by `controllers/etl4_benchmarking.py`, and the results are saved in the `olap_query_results` folder in the root directory.

Tables IV–VI report the result sets returned by Queries 2, 4 and 5.

TABLE IV
Query 2: MODIS vs. VIIRS Sensor Comparison

Sensor	Satellite	Detections	Avg FRP (MW)	Avg brightness (K)	Avg confidence
VIIRS	N20	731,630	4.65	327.73	46.7
VIIRS	N	574,152	4.75	328.35	46.9
MODIS	Aqua	105,012	23.00	324.73	62.7
MODIS	Terra	70,776	13.61	315.07	63.7

TABLE V
Query 4: Daytime vs. Nighttime Detections

Period	Detections	Avg FRP (MW)	Avg confidence
Daytime (D)	923,349	8.75	47.1
Nighttime (N)	558,221	2.55	51.4

TABLE VI
Query 5: Quarterly Fire Density by Confidence Level

Quarter	Fire count	High conf.	Nominal conf.	Low conf.
2024 Q1	297,618	12,294	226,925	58,399
2024 Q2	964,362	34,390	790,561	139,411
2024 Q3	110,130	1,163	107,217	1,750
2024 Q4	44,077	468	40,616	2,993
2025 Q1	65,383	2,191	59,654	3,538

V. Mining Preliminary Pipeline

A. Data Preparation

An ML schema is created by `init-scripts/05_create_ml_schema.sql`. The mining pipeline is implemented in `dags/ml_mining.py` using pandas, scikit-learn and SciPy, reading from the warehouse through psycopg2.

B. Feature Engineering Strategy

Two mining tasks are prepared:

- **HDBSCAN spatio-temporal clustering:** groups discrete satellite detection points into cohesive physical wildfire event perimeters.
- **Isolation Forest anomaly detection:** trained on the thermal radiative metrics `frp`, `brightness_kelvin` and `confidence` to flag high-intensity illegal slash-and-burn activities and abnormal thermal spikes.